

# Experiment 4A: CUDA Vector Addition

This guide explains how to perform Vector Addition using CUDA. The guide includes corrected CUDA code, installation steps, compilation steps, sample input/output, and viva questions.

**Problem Statement:** Write a CUDA program for addition of two large vectors.

**Important:** CUDA requires an NVIDIA GPU and CUDA Toolkit installation. This program will not run using AMD or Intel GPUs.

## Correct CUDA Program Code

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

// CUDA Kernel
__global__ void vectorAdd(float *A, float *B, float *C, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N)
    {
        C[i] = A[i] + B[i];
    }
}

int main()
{
    int N;

    printf("Enter number of elements: ");
    scanf("%d", &N);

    int size = N * sizeof(float);

    // Host memory
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);
    float *h_C = (float*)malloc(size);

    // Input first vector
    printf("Enter elements of first vector:\n");

    for (int i = 0; i < N; i++)
    {
        scanf("%f", &h_A[i]);
    }

    // Input second vector
    printf("Enter elements of second vector:\n");

    for (int i = 0; i < N; i++)
    {
        scanf("%f", &h_B[i]);
    }

    // Device memory
    float *d_A, *d_B, *d_C;

    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    // Copy Host to Device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    // Thread configuration
```

```

int threadsPerBlock = 256;

int blocksPerGrid =
(N + threadsPerBlock - 1) / threadsPerBlock;

// Launch Kernel
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>
(d_A, d_B, d_C, N);

cudaDeviceSynchronize();

// Copy result back
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

// Display result
printf("\nResultant Vector:\n");

for (int i = 0; i < N; i++)
{
    printf("%.2f ", h_C[i]);
}

printf("\n");

// Free device memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);

// Free host memory
free(h_A);
free(h_B);
free(h_C);

return 0;
}

```

## How to Run on Windows

1. Install NVIDIA CUDA Toolkit.
2. Open Command Prompt.
3. Save file as vector\_addition.cu.
4. Go to file location using cd command.
5. Compile using: nvcc vector\_addition.cu -o vector\_addition
6. Run using: vector\_addition

## How to Run on Linux / Ubuntu

Compile and run:

```

sudo apt update

Install NVIDIA Driver

Install CUDA Toolkit

nvcc vector_addition.cu -o vector_addition

./vector_addition

```

## Sample Input

```

Enter number of elements: 5

Enter elements of first vector:
1

```

2  
3  
4  
5

Enter elements of second vector:

5  
4  
3  
2  
1

## Expected Output

Resultant Vector:  
6.00 6.00 6.00 6.00 6.00

## Important Viva Questions

| Question                   | Answer                                        |
|----------------------------|-----------------------------------------------|
| What is CUDA?              | CUDA is NVIDIA's parallel computing platform. |
| What is GPU?               | GPU performs many operations simultaneously.  |
| What is kernel?            | A function executed on GPU.                   |
| Why use CUDA?              | To speed up parallel computations.            |
| What does __global__ mean? | It defines a GPU kernel function.             |